30.2 插件钩子（Plugin Hooks）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~140k tokens，输出 ~8k tokens（本节）

上一节介绍的内部钩子（Internal Hooks）运行在 Gateway 进程内部，通过事件键注册和触发。本节聚焦 OpenClaw 的另一套钩子体系——**插件钩子（Plugin Hooks）**，它由插件系统驱动，提供了更结构化的生命周期拦截能力，以及将外部 HTTP 请求映射为 Agent 动作的网关钩子映射机制。

30.2.1 钩子映射（ `src/gateway/hooks-mapping.ts` ）

设计场景

想象这样一个场景：Gmail 收到新邮件时，你希望 OpenClaw 的 Agent 自动处理。传统做法需要写一个中间服务来桥接 Gmail Webhook 和 Agent API。OpenClaw 的**钩子映射（Hook Mapping）**机制直接内置了这个桥接能力——它接收外部 HTTP 请求，按规则匹配后转换为 Agent 可理解的动作。

整体架构

```
外部系统（Gmail/GitHub/自定义服务）
        │
        ▼ HTTP POST /hooks/gmail
┌──────────────────────────────┐
│   Gateway HTTP 端点          │
│   ① Token 鉴权               │
│   ② 路径/来源匹配            │
│   ③ 模板渲染                 │
│   ④ Transform 转换（可选）   │
│   ⑤ 生成 HookAction          │
└──────────────────────────────┘
        │
        ▼
   wake（唤醒心跳）或 agent（发送消息给 Agent）
```

钩子端点配置

Gateway 钩子端点通过配置启用，必须设置鉴权 Token：

```
// src/gateway/hooks.ts（简化）

export function resolveHooksConfig(cfg: OpenClawConfig): HooksConfigResolved | null {
  if (cfg.hooks?.enabled !== true) return null;

  const token = cfg.hooks?.token?.trim();
  if (!token) throw new Error("hooks.enabled requires hooks.token");

  const basePath = cfg.hooks?.path?.trim() || "/hooks";
  const maxBodyBytes = cfg.hooks?.maxBodyBytes ?? 256 * 1024;  // 256KB
  const mappings = resolveHookMappings(cfg.hooks);

  return { basePath, token, maxBodyBytes, mappings };
}
```

Token 从请求中提取，支持两种方式：

方式	HTTP 头	示例
Bearer Token	`Authorization: Bearer <token>`	标准 OAuth 风格
自定义头	`X-OpenClaw-Token: <token>`	简化集成

映射规则与预设

映射规则定义了"什么请求触发什么动作"。每条规则包含匹配条件和动作模板：

```ts
// src/gateway/hooks-mapping.ts

export type HookMappingResolved = {
  id: string;                  // 规则 ID
  matchPath?: string;          // 匹配 URL 路径（如 "gmail"）
  matchSource?: string;        // 匹配 payload.source 字段
  action: "wake" | "agent";    // 动作类型
  wakeMode?: "now" | "next-heartbeat";
  messageTemplate?: string;    // 消息模板（Mustache 风格）
  sessionKey?: string;         // 会话键模板
  transform?: {                // 可选的自定义转换模块
    modulePath: string;
    exportName?: string;
  };
  // ...更多字段
};
```

OpenClaw 内置了 Gmail 预设映射：

```ts
const hookPresetMappings: Record<string, HookMappingConfig[]> = {
  gmail: [{
    id: "gmail",
    match: { path: "gmail" },
    action: "agent",
    wakeMode: "now",
    name: "Gmail",
    sessionKey: "hook:gmail:{{messages[0].id}}",
    messageTemplate:
      "New email from {{messages[0].from}}\n" +
      "Subject: {{messages[0].subject}}\n" +
      "{{messages[0].snippet}}\n{{messages[0].body}}",
  }],
};
```

用户也可以在配置中定义自定义映射规则：

```
{
  "hooks": {
    "enabled": true,
    "token": "my-secret-token",
    "presets": ["gmail"],
    "mappings": [{
      "id": "github-pr",
      "match": { "path": "github", "source": "github" },
      "action": "agent",
      "sessionKey": "hook:github:{{pull_request.id}}",
      "messageTemplate": "PR #{{pull_request.number}}: {{pull_request.title}}"
    }]
  }
}
```

匹配与渲染

HTTP 请求到达钩子端点时， `applyHookMappings` 按顺序尝试匹配：

```
// src/gateway/hooks-mapping.ts（简化）

export async function applyHookMappings(
  mappings: HookMappingResolved[],
  ctx: HookMappingContext,
): Promise<HookMappingResult | null> {
  for (const mapping of mappings) {
    // 1. 路径匹配
    if (mapping.matchPath && mapping.matchPath !== normalizeMatchPath(ctx.path)) contin
    // 2. 来源匹配
    if (mapping.matchSource && ctx.payload.source !== mapping.matchSource) continue;

    // 3. 构建基础动作（模板渲染）
    const base = buildActionFromMapping(mapping, ctx);

    // 4. 可选：执行 Transform 模块
    if (mapping.transform) {
      const transform = await loadTransform(mapping.transform);
      const override = await transform(ctx);
      if (override === null) return { ok: true, action: null, skipped: true };
    }

    // 5. 合并基础动作和 Transform 结果
    return mergeAction(base.action, override, mapping.action);
  }
  return null;  // 无匹配
}
```

模板引擎

消息模板使用类 Mustache 的 `{{expr}}` 语法，支持以下表达式：

```
// src/gateway/hooks-mapping.ts

function resolveTemplateExpr(expr: string, ctx: HookMappingContext) {
  if (expr === "path")    return ctx.path;           // 请求路径
  if (expr === "now")     return new Date().toISOString(); // 当前时间
  if (expr.startsWith("headers."))  return getByPath(ctx.headers, ...);
  if (expr.startsWith("query."))    return getByPath(queryParams, ...);
  if (expr.startsWith("payload."))  return getByPath(ctx.payload, ...);
  return getByPath(ctx.payload, expr);  // 默认在 payload 中查找
}
```

`getByPath` 支持点号和方括号语法（如 `messages[0].from`），可以深度访问嵌套的 JSON 数据。

动作类型

匹配成功后生成两种动作之一：

动作	用途	效果
`wake`	唤醒 Agent 心跳	发送一段文本触发 Agent 心跳处理
`agent`	发送消息给 Agent	创建一个新的 Agent 消息，包含 sessionKey、通道、模型等参数

`agent` 动作是最常用的——它等价于从某个通道发送一条消息给 Agent，但消息内容由外部系统的 Webhook 数据填充。

Transform 自定义转换

对于复杂的数据转换需求，模板语法可能不够用。OpenClaw 支持加载自定义 Transform 模块来完全控制映射逻辑：

```
// 自定义 Transform 模块示例
export default function transform(ctx: HookMappingContext) {
  const payload = ctx.payload;
  // 过滤不需要处理的事件
  if (payload.action !== "opened") return null;  // 返回 null 跳过

  return {
    message: `New issue: ${payload.issue.title}`,
    sessionKey: `hook:github:issue-${payload.issue.id}`,
  };
}
```

Transform 函数接收完整的请求上下文，返回部分动作字段（与基础模板渲染结果合并），或返回 `null` 跳过此请求。Transform 模块带有缓存——同一模块路径只会被 `import()` 一次。

30.2.2 Agent 生命周期钩子： `before_agent_start` / `agent_end`

从这一小节开始，我们进入**插件钩子系统（Plugin Hook System）**的领域。与内部钩子的发布/订阅模式不同，插件钩子通过插件注册表（Plugin Registry）管理，由 `createHookRunner` 创建的运行器统一调度。

插件钩子总览

OpenClaw 定义了 14 种插件钩子，覆盖五个生命周期域：

```
// src/plugins/types.ts

export type PluginHookName =
  | "before_agent_start" | "agent_end"                    // Agent 域
  | "before_compaction"  | "after_compaction"              // 压缩域
  | "message_received"   | "message_sending" | "message_sent"  // 消息域
  | "before_tool_call"   | "after_tool_call" | "tool_result_persist"  // 工具域
  | "session_start"      | "session_end"                   // 会话域
  | "gateway_start"      | "gateway_stop";                 // 网关域
```

执行模型：Void vs Modifying

插件钩子有两种执行模式，对应不同的使用场景：

```
// src/plugins/hooks.ts（简化）

// 模式一：Void Hook —— 并行执行，火即忘
async function runVoidHook(hookName, event, ctx): Promise<void> {
  const hooks = getHooksForName(registry, hookName);
  // 所有处理器并行执行
  await Promise.all(hooks.map(hook => hook.handler(event, ctx)));
}

// 模式二：Modifying Hook —— 顺序执行，结果合并
async function runModifyingHook(hookName, event, ctx, merge): Promise<TResult> {
  const hooks = getHooksForName(registry, hookName);
  let result;
  // 按优先级顺序逐个执行，合并结果
  for (const hook of hooks) {
    const next = await hook.handler(event, ctx);
    result = merge ? merge(result, next) : next;
  }
  return result;
}
```

模式	执行方式	返回值	适用场景
Void	`Promise.all` （并行）	无	观察型钩子（日志、统计）
Modifying	顺序串行	可返回修改结果	拦截型钩子（修改、阻止）

> **衍生解释**：这种双模式设计在中间件架构中很常见。Express.js 的中间件是纯顺序的（每个 `next()` 调用下一个）；Koa 的洋葱模型也是顺序的。OpenClaw 增加了并行模式——对于不需要互相影响的观察型钩子，并行执行能显著提高性能。

before_agent_start

`before_agent_start` 在 Agent 开始处理请求前触发，允许插件注入额外的系统提示词或上下文：

```
// src/plugins/types.ts

export type PluginHookBeforeAgentStartEvent = {
  prompt: string;          // 用户提示词
  messages?: unknown[];    // 历史消息
};

export type PluginHookBeforeAgentStartResult = {
  systemPrompt?: string;      // 替换系统提示词
  prependContext?: string;    // 在上下文前追加内容
};
```

这是一个 **Modifying Hook**——多个插件的结果会被合并：

```
// src/plugins/hooks.ts

async function runBeforeAgentStart(event, ctx) {
  return runModifyingHook("before_agent_start", event, ctx,
    (acc, next) => ({
      systemPrompt: next.systemPrompt ?? acc?.systemPrompt,
      prependContext: acc?.prependContext && next.prependContext
        ? `${acc.prependContext}\n\n${next.prependContext}`  // 拼接
        : (next.prependContext ?? acc?.prependContext),
    }),
  );
}
```

合并策略是： `systemPrompt` 后者覆盖前者（最后一个插件说了算）， `prependContext` 则追加拼接（所有插件的前置上下文都保留）。

agent_end

`agent_end` 在 Agent 完成处理后触发，是一个 **Void Hook**（并行执行、无返回值）：

```
export type PluginHookAgentEndEvent = {
  messages: unknown[];      // 完整的对话消息列表
  success: boolean;         // 是否成功完成
  error?: string;           // 错误信息（如有）
  durationMs?: number;      // 处理耗时
};
```

典型用途包括：对话质量分析、使用量统计上报、异常监控告警等。

30.2.3 工具生命周期钩子： `before_tool_call` / `after_tool_call` / `tool_result_persist`

工具钩子是插件钩子中最精细的一组，覆盖了工具调用的完整生命周期。

before_tool_call

在 Agent 发起工具调用之前触发。这是一个 **Modifying Hook**，插件可以修改参数甚至阻止调用：

```
export type PluginHookBeforeToolCallEvent = {
  toolName: string;                    // 工具名称
  params: Record<string, unknown>;     // 调用参数
};

export type PluginHookBeforeToolCallResult = {
  params?: Record<string, unknown>;    // 修改后的参数
  block?: boolean;                     // 是否阻止调用
  blockReason?: string;                // 阻止原因
};
```

合并策略：

```
(acc, next) => ({
  params: next.params ?? acc?.params,
  block: next.block ?? acc?.block,
  blockReason: next.blockReason ?? acc?.blockReason,
})
```

任何一个插件返回 `block: true` ，工具调用就会被阻止——这是安全策略的关键拦截点。比如，一个安全插件可以检查 `bash` 工具的参数，阻止执行危险命令：

```
// 假设的安全插件
api.on("before_tool_call", (event) => {
  if (event.toolName === "bash" && event.params.command?.includes("rm -rf /")) {
    return { block: true, blockReason: "Dangerous command blocked" };
  }
});
```

after_tool_call

在工具调用完成后触发，是一个 **Void Hook**（并行、观察型）：

```
export type PluginHookAfterToolCallEvent = {
  toolName: string;                    // 工具名称
  params: Record<string, unknown>;     // 调用参数
  result?: unknown;                    // 调用结果
  error?: string;                      // 错误信息
  durationMs?: number;                 // 执行耗时
};
```

典型用途：工具使用统计、性能监控、错误追踪。

tool_result_persist（同步钩子）

`tool_result_persist` 是整个钩子系统中**唯一的同步钩子**——它在工具结果写入会话记录时触发，必须同步返回：

```
// src/plugins/hooks.ts

function runToolResultPersist(event, ctx): PluginHookToolResultPersistResult | undefine
  const hooks = getHooksForName(registry, "tool_result_persist");
  let current = event.message;

  for (const hook of hooks) {
    const out = hook.handler({ ...event, message: current }, ctx);

    // 防御：拒绝异步处理器
    if (out && typeof out.then === "function") {
      logger?.warn(`tool_result_persist handler returned a Promise; ignored.`);
      continue;
    }

    const next = out?.message;
    if (next) current = next;  // 管道式传递
  }
  return { message: current };
}
```

设计为同步的原因是：此钩子运行在**会话记录追加的热路径**上——会话 JSONL 文件的写入是同步操作，如果这里引入异步等待，会破坏写入的原子性和顺序性。

这个钩子采用**管道模式**——每个处理器接收上一个处理器的输出作为输入，最终结果是链式传递的最后一版消息。典型用途：在持久化前过滤敏感信息、压缩大型工具输出、添加元数据标注。

> **衍生解释**：管道模式（Pipeline Pattern）是一种数据处理模式，数据依次通过一系列处理阶段，每个阶段接收前一阶段的输出并产生新的输出。Unix 的管道操作符 `|` 是最经典的例子：`cat file | grep error | sort`。这里的 `tool_result_persist` 钩子链就是一个同步的内存管道。

30.2.4 消息钩子：`message_received` / `message_sending` / `message_sent`

消息钩子拦截用户与 Agent 之间的消息流，覆盖消息的"进-处理-出"三个阶段：

```
用户消息 ──→ message_received ──→ Agent 处理 ──→ message_sending ──→ 发送 ──→ message_sent
              (观察)                              (可修改/取消)          (观察)
```

message_received

用户消息到达时触发，是一个 **Void Hook**：

```
export type PluginHookMessageReceivedEvent = {
  from: string;                          // 发送者标识
  content: string;                       // 消息内容
  timestamp?: number;                    // 时间戳
  metadata?: Record<string, unknown>;    // 通道特定的元数据
};

export type PluginHookMessageContext = {
  channelId: string;      // 通道 ID (telegram/discord/webchat 等)
  accountId?: string;     // 账户 ID
  conversationId?: string; // 会话 ID
};
```

用途：消息接收统计、内容审计日志、用户行为分析。

message_sending

Agent 回复即将发送前触发，是一个 **Modifying Hook**——插件可以修改内容甚至取消发送：

```
export type PluginHookMessageSendingEvent = {
  to: string;                 // 接收者
  content: string;            // 消息内容
  metadata?: Record<string, unknown>;
};

export type PluginHookMessageSendingResult = {
  content?: string;           // 修改后的内容
  cancel?: boolean;           // 是否取消发送
};
```

合并策略：

```
(acc, next) => ({
  content: next.content ?? acc?.content,
  cancel: next.cancel ?? acc?.cancel,
})
```

这个钩子给了插件极大的控制权：

- **内容过滤**：在发送前替换敏感信息或不恰当内容
- **格式转换**：将 Markdown 转为特定通道支持的格式
- **发送拦截**：满足特定条件时阻止消息发出

message_sent

消息实际发送后触发，是一个 **Void Hook**：

```
export type PluginHookMessageSentEvent = {
  to: string;
  content: string;
  success: boolean;     // 发送是否成功
  error?: string;       // 失败原因
};
```

用途：发送成功率监控、失败重试触发、消息归档。

30.2.5 网关钩子： `gateway_start` / `gateway_stop`

网关钩子在 Gateway 服务器的启动和停止时触发，是最外层的生命周期钩子。

gateway_start

Gateway HTTP 服务器启动后触发，是一个 **Void Hook**：

```typescript
export type PluginHookGatewayStartEvent = {
  port: number;    // 监听端口
};

export type PluginHookGatewayContext = {
  port?: number;
};
```

用途：初始化外部连接（数据库、消息队列）、注册服务发现、发送启动通知。

gateway_stop

Gateway 即将关闭时触发：

```typescript
export type PluginHookGatewayStopEvent = {
  reason?: string;   // 关闭原因
};
```

用途：清理资源、断开外部连接、发送关闭通知、刷新缓冲区。

钩子运行器工厂

所有插件钩子通过 `createHookRunner` 统一创建运行器：

```
// src/plugins/hooks.ts

export function createHookRunner(registry: PluginRegistry, options = {}) {
  const logger = options.logger;
  const catchErrors = options.catchErrors ?? true;  // 默认捕获错误

  return {
    // Agent 钩子
    runBeforeAgentStart, runAgentEnd,
    runBeforeCompaction, runAfterCompaction,
    // 消息钩子
    runMessageReceived, runMessageSending, runMessageSent,
    // 工具钩子
    runBeforeToolCall, runAfterToolCall, runToolResultPersist,
    // 会话钩子
    runSessionStart, runSessionEnd,
    // 网关钩子
    runGatewayStart, runGatewayStop,
    // 工具方法
    hasHooks,      // 检查某钩子是否有注册
    getHookCount,  // 获取某钩子的注册数量
  };
}
```

运行器的几个关键设计决策：

1. **默认容错**： `catchErrors` 默认为 `true` ，一个插件的钩子失败不会影响其他插件和核心流程。
2. **优先级排序**：处理器按 `priority` 降序排列——高优先级先执行。对于 Modifying Hook，这意味着高优先级的插件可以先设定默认值，低优先级的可以覆盖。
3. **按需检查**： `hasHooks` 让调用方可以在不调用运行器的情况下判断是否有注册钩子，避免不必要的事件对象构建开销。

插件钩子注册

插件通过 `OpenClawPluginApi.on()` 方法注册钩子处理器：

```
// 插件代码示例
export default function myPlugin(api: OpenClawPluginApi) {
  // 注册 Agent 生命周期钩子
  api.on("before_agent_start", async (event, ctx) => {
    return { prependContext: "Always be concise." };
  });

  // 注册工具钩子（带优先级）
  api.on("before_tool_call", async (event, ctx) => {
    if (event.toolName === "bash") {
      console.log(`[audit] bash: ${event.params.command}`);
    }
  }, { priority: 100 });

  // 注册网关钩子
  api.on("gateway_start", async (event) => {
    console.log(`Gateway started on port ${event.port}`);
  });
}
```

每个注册会创建一个 `PluginHookRegistration` 记录，存储在插件注册表中：

```
export type PluginHookRegistration<K extends PluginHookName> = {
  pluginId: string;           // 来源插件 ID
  hookName: K;                // 钩子名称
  handler: PluginHookHandlerMap[K];  // 处理器函数
  priority?: number;          // 优先级（越高越先执行）
  source: string;             // 来源描述
};
```

两套钩子系统的对比

至此，我们已经完整介绍了 OpenClaw 的两套钩子系统。以下是它们的关键差异：

维度	内部钩子（Internal Hooks）	插件钩子（Plugin Hooks）
来源	HOOK.md + handler.ts 文件	插件代码中 `api.on()` 注册
管理方式	目录发现 + 配置控制	插件注册表
事件命名	`type:action` 字符串键	枚举类型 `PluginHookName`

维度	内部钩子（Internal Hooks）	插件钩子（Plugin Hooks）
执行模式	全部顺序串行	Void（并行）/ Modifying（顺序）
返回值	无（通过 `event.messages` 回推）	Modifying Hook 可返回修改结果
典型用途	文件系统级扩展（自定义钩子目录）	编程级扩展（插件代码）
安装方式	`openclaw hooks install`	插件系统自动加载
错误处理	try/catch 隔离	可配置（默认捕获）

两套系统互不冲突，可以同时使用。内部钩子更适合"运维级"的轻量扩展（如审计日志、会话保存），插件钩子更适合"开发级"的深度集成（如安全拦截、内容转换、外部系统联动）。

本节小结

1. **钩子映射**是 Gateway 内置的 HTTP→Agent 桥接机制，通过路径/来源匹配和模板渲染将外部 Webhook 请求转换为 Agent 动作，支持预设（Gmail）和自定义映射规则，以及 Transform 模块进行复杂转换。
2. **插件钩子系统**定义了 14 种钩子，覆盖 Agent、消息、工具、会话、网关五个生命周期域，由 `createHookRunner` 工厂统一调度。
3. **Agent 钩子**中， `before_agent_start` 允许注入系统提示词（顺序合并）， `agent_end` 用于完成后的观察分析（并行执行）。
4. **工具钩子**提供了最精细的控制——`before_tool_call` 可以修改参数或阻止调用， `tool_result_persist` 是唯一的同步管道钩子，在会话记录的热路径上运行。
5. **消息钩子**拦截消息的进-出流程，其中 `message_sending` 是唯一可以修改或取消消息的拦截点。
6. **两套钩子系统**（内部 + 插件）互补共存——内部钩子面向文件系统级扩展，插件钩子面向编程级深度集成，共同构成了 OpenClaw 完整的可扩展性基础。